

# FIM 548

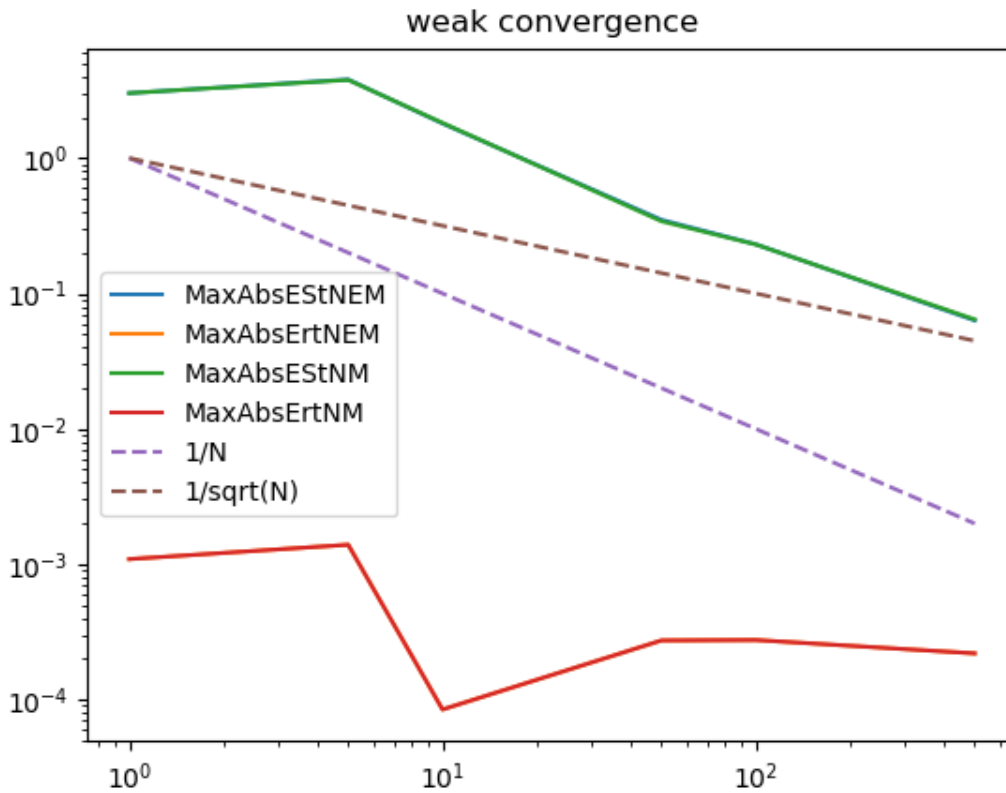
## HW 2

March 11, 2026

Jacob Attia, Clay Ewing, Zach Wiebe

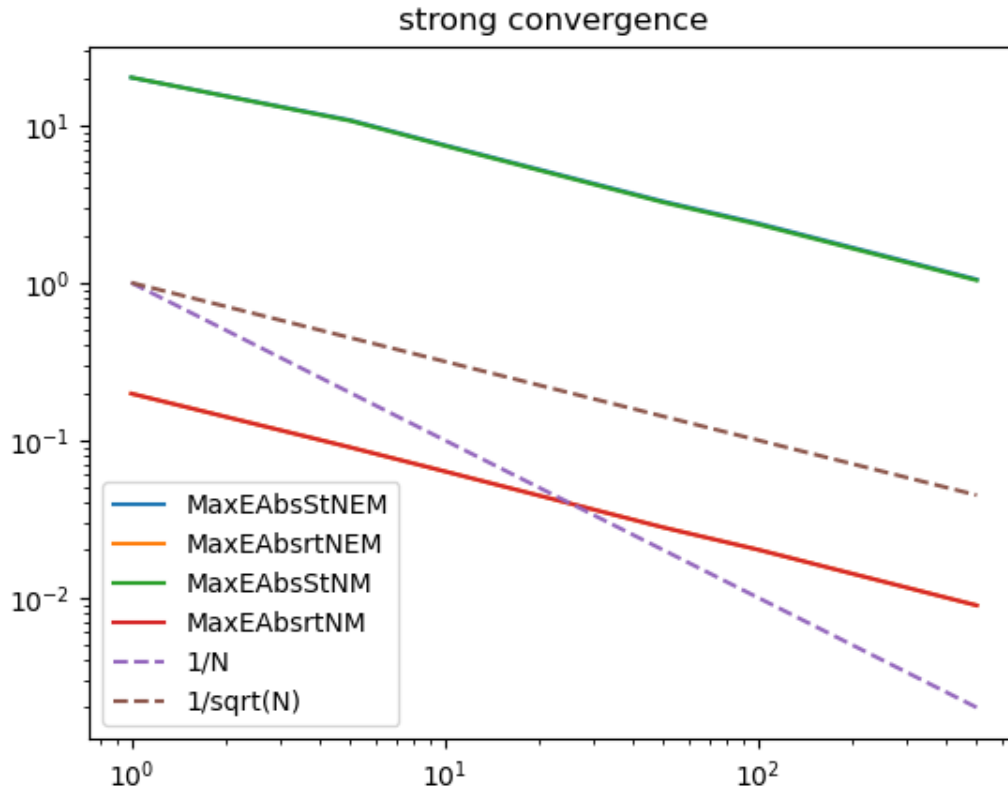
---

1. We used 10000 simulations per expected value and  $N$  values of 1,5,10,50,100,500.  
Looking at the first mode of convergence,



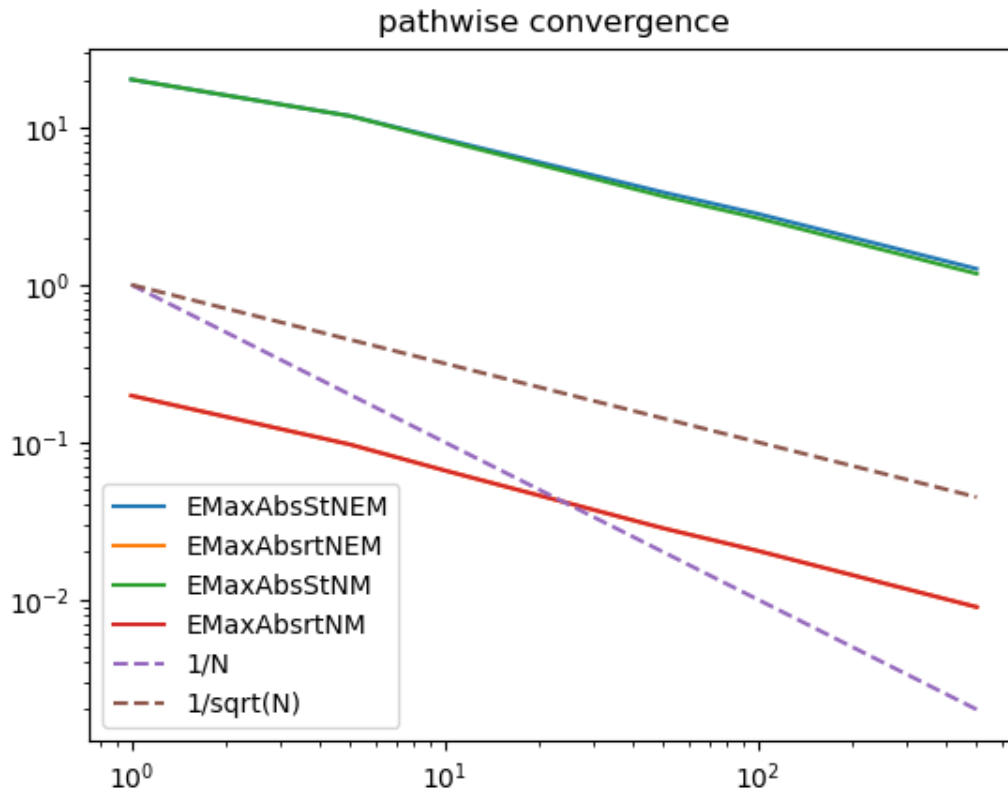
We see that both approximation methods for  $S_t$  entirely overlap and converge roughly like  $\frac{1}{N}$ , and both approximation methods for  $r_t$  entirely overlap and converge roughly like  $\frac{1}{\sqrt{N}}$ .

Looking at the second mode of convergence,



We see that both approximation methods for  $S_t$  entirely overlap and converge roughly like  $\frac{1}{\sqrt{N}}$ , and both approximation methods for  $r_t$  entirely overlap and converge roughly like  $\frac{1}{\sqrt{N}}$ .

Looking at the third mode of convergence,



We see that the Milstein method approximation for  $S_t$  grows slightly more accurate than the Euler-Maruyama approximation for large  $N$ , but both still roughly converge like  $\frac{1}{\sqrt{N}}$ . We also see that both approximation methods for  $r_t$  entirely overlap and converge roughly like  $\frac{1}{\sqrt{N}}$ .

Here is the code:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Constants
5  alpha = 3
6  sigma = 0.25
7  rbar = 0.05
8  rnaught = 0.05
9  snaught = 100
10 mu = 0.15
11 Ns = [1,5,10,50,100,500]#,1000]
12 N = max(Ns)
13 T = 1
14
15 #t = np.linspace(0,T,N)
16
17 # Expected value of a random variable
18 def expectation(sims: int, expectee, N, B):
19     trials = []
20     i = 0
21     while i < sims:
22         trials.append(expectee(N,B[i]))
23         i += 1
24
25     return np.average(trials,axis=0)
26
27 # Brownian motion seed
28 def Bt(N: int):
29     Z = np.random.normal(0,1,N)
30     dt = np.ones(N) * T/N
31     return np.cumsum(Z*np.sqrt(dt))
32
33 # rt exact simulation
34 def rt(N: int, B: list):
35     integral = []
36     j = 0
37     while j < N:
38         nextInt = -B[0]
39         i = 1
40         while i <= j:
41             nextInt += np.exp(-alpha*(t[j]-t[i]))*(B[i]-B[i-1])
42             i += 1
43         integral.append(nextInt)
44         j += 1
45     return rnaught*np.exp(-alpha*t)+rbar*(1-np.exp(-alpha*t))+np.multiply(sigma,integral)
46
47 # rtN Euler-Maruyama
48 def rtNEM(N: int, B: list):
49     rtN = [rnaught]
50     i = 1
51     while i < N:
52         rtN.append((1-alpha*T/N)*rtN[i-1]+alpha*rbar*T/N+sigma*(B[i]-B[i-1]))
53         i += 1
54     return rtN
55
56 # rtN Milstein
57 def rtNM(N: int, B: list):
58     rtN = [rnaught]
59     i = 1
60     while i < N:
61         rtN.append(rtN[i-1]+alpha*(rbar-rtN[i-1])*T/N+sigma*(B[i]-B[i-1]))
62         i += 1
63     return rtN
64
65 # Absolute difference of rtNEM and rt
66 def absrtNEMminusrt(N: int, B: list):
67     return np.abs(np.subtract(rtNEM(N,B),rt(N,B)))
68
69 # Max_t of absolute difference of rtNEM and rt
70 def maxAbsrtNEMminusrt(N: int, B: list):
71     return max(np.abs(np.subtract(rtNEM(N,B),rt(N,B))))
72
73 # Absolute difference of rtNM and rt
74 def absrtNMminusrt(N: int, B: list):
75     return np.abs(np.subtract(rtNM(N,B),rt(N,B)))
76

```

```

77 # Max_t of absolute difference of rtNM and rt
78 def maxAbsrtNMminusrt(N: int, B: list):
79     return max(np.abs(np.subtract(rtNM(N,B),rt(N,B))))
80
81 # St exact simulation
82 def St(N: int, B: list): # St is geometric Brownian motion
83     return snaught*np.exp((mu-sigma**2/2)*t+sigma*B)
84
85 # StN Euler-Maruyama
86 def StNEM(N: int, B: list):
87     StN = [snaught]
88     i = 1
89     while i < N:
90         StN.append(StN[i-1]*(1+mu*T/N+sigma*(B[i]-B[i-1])))
91         i += 1
92     return StN
93
94 # StN Milstein
95 def StNM(N: int, B: list):
96     StN = [snaught]
97     i = 1
98     while i < N:
99         StN.append(StN[i-1]+StN[i-1]*mu*T/N+sigma*StN[i-1]*(B[i]-B[i-1])+0.5*sigma**2*StN[i-1]*((B[i]-B[i-1])**2-T/N))
100        i += 1
101    return StN
102
103 # Absolute difference of StNEM and St
104 def absStNEMminusSt(N: int, B: list):
105     return np.abs(np.subtract(StNEM(N,B),St(N,B)))
106
107 # Max_t of absolute difference of StNEM and St
108 def maxAbsStNEMminusSt(N: int, B: list):
109     return max(np.abs(np.subtract(StNEM(N,B),St(N,B))))
110
111 # Absolute difference of StNM and St
112 def absStNMminusSt(N: int, B: list):
113     return np.abs(np.subtract(StNM(N,B),St(N,B)))
114
115 # Max_t of absolute difference of StNM and St
116 def maxAbsStNMminusSt(N: int, B: list):
117     return max(np.abs(np.subtract(StNM(N,B),St(N,B))))
118
119 # Run simulations from functions defined above
120 sims = 10000
121 Ert = []
122 ErtNEM = []
123 ErtNM = []
124 MaxAbsErtNEM = []
125 MaxEAbsrtNEM = []
126 EMaxAbsrtNEM = []
127 MaxAbsErtNM = []
128 MaxEAbsrtNM = []
129 EMaxAbsrtNM = []
130
131 ESt = []
132 EStNEM = []
133 EStNM = []
134 MaxAbsEStNEM = []
135 MaxEAbsStNEM = []
136 EMaxAbsStNEM = []
137 MaxAbsEStNM = []
138 MaxEAbsStNM = []
139 EMaxAbsStNM = []
140 i = 0
141 while i < len(Ns):
142     j = 0
143     B = []
144     while j < sims:
145         B.append(Bt(Ns[i]))
146         j += 1
147     t = np.linspace(0,1,Ns[i])
148
149     Ert.append(expectation(sims,rt,Ns[i],B))
150     ESt.append(expectation(sims,St,Ns[i],B))

```

```

151     ErtNEM.append(expectation(sims,rtNEM,Ns[i],B))
152     EstNEM.append(expectation(sims,StNEM,Ns[i],B))
153     MaxAbsErtNEM.append(max(np.abs(np.subtract(Ert[i],ErtNEM[i]))))
154     MaxAbsEstNEM.append(max(np.abs(np.subtract(Est[i],EstNEM[i]))))
155     ErtNM.append(expectation(sims,rtNM,Ns[i],B))
156     EstNM.append(expectation(sims,StNM,Ns[i],B))
157     MaxAbsErtNM.append(max(np.abs(np.subtract(Ert[i],ErtNM[i]))))
158     MaxAbsEstNM.append(max(np.abs(np.subtract(Est[i],EstNM[i]))))
159
160     MaxEabsrtNEM.append(max(expectation(sims,absrtNEMminusrt,Ns[i],B)))
161     MaxEabsStNEM.append(max(expectation(sims,absStNEMminusSt,Ns[i],B)))
162     MaxEabsrtNM.append(max(expectation(sims,absrtNMminusrt,Ns[i],B)))
163     MaxEabsStNM.append(max(expectation(sims,absStNMminusSt,Ns[i],B)))
164
165     EMaxAbsrtNEM.append(expectation(sims,maxAbsrtNEMminusrt,Ns[i],B))
166     EMaxAbsStNEM.append(expectation(sims,maxAbsStNEMminusSt,Ns[i],B))
167     EMaxAbsrtNM.append(expectation(sims,maxAbsrtNMminusrt,Ns[i],B))
168     EMaxAbsStNM.append(expectation(sims,maxAbsStNMminusSt,Ns[i],B))
169     i += 1
170
171     # Create plots
172     plt.plot(Ns,MaxAbsEstNEM,label="MaxAbsEstNEM")
173     plt.plot(Ns,MaxAbsErtNEM,label="MaxAbsErtNEM")
174     plt.plot(Ns,MaxAbsEstNM,label="MaxAbsEstNM")
175     plt.plot(Ns,MaxAbsErtNM,label="MaxAbsErtNM")
176     plt.plot(Ns,np.divide(1,Ns),ls='--',label="1/N")
177     plt.plot(Ns,np.divide(1,np.sqrt(Ns)),ls='--',label="1/sqrt(N)")
178
179     plt.xscale('log')
180     plt.yscale('log')
181     plt.legend()
182     plt.title("weak convergence")
183     plt.show()
184
185
186
187     plt.plot(Ns,MaxEabsStNEM,label="MaxEabsStNEM")
188     plt.plot(Ns,MaxEabsrtNEM,label="MaxEabsrtNEM")
189     plt.plot(Ns,MaxEabsStNM,label="MaxEabsStNM")
190     plt.plot(Ns,MaxEabsrtNM,label="MaxEabsrtNM")
191     plt.plot(Ns,np.divide(1,Ns),ls='--',label="1/N")
192     plt.plot(Ns,np.divide(1,np.sqrt(Ns)),ls='--',label="1/sqrt(N)")
193
194     plt.xscale('log')
195     plt.yscale('log')
196     plt.legend()
197     plt.title("strong convergence")
198     plt.show()
199
200
201
202     plt.plot(Ns,EMaxAbsStNEM,label="EMaxAbsStNEM")
203     plt.plot(Ns,EMaxAbsrtNEM,label="EMaxAbsrtNEM")
204     plt.plot(Ns,EMaxAbsStNM,label="EMaxAbsStNM")
205     plt.plot(Ns,EMaxAbsrtNM,label="EMaxAbsrtNM")
206     plt.plot(Ns,np.divide(1,Ns),ls='--',label="1/N")
207     plt.plot(Ns,np.divide(1,np.sqrt(Ns)),ls='--',label="1/sqrt(N)")
208
209     plt.xscale('log')
210     plt.yscale('log')
211     plt.legend()
212     plt.title("pathwise convergence")
213     plt.show()

```